Mini Projet : **Gestion du déséquilibre de classes pour l'analyse de sentiments en dialecte Algérien**

Année universitaire : 2025 — 2026
Niveau : Master 1 DS & NLP — Semestre 2
Module : Machine Learning
Enseignant(e) : Dr. Soraya Cheriguene
Date de distribution : 08 Mars 2026
Date de soutenance : 26 Avril 2026
Durée estimée : 4 à 6 semaines
Mode de travail : En binôme ou monôme

1. Contexte :

Le dialecte algérien (darija) est une langue peu dotée en ressources numériques, ce qui pose des défis importants pour les systèmes de Traitement Automatique du Langage Naturel (TALN). L'un des problèmes les plus fréquents dans ce domaine est le déséquilibre entre les classes : certains sentiments ou certaines émotions sont très peu représentés dans les corpus disponibles, ce qui biaise les modèles d'apprentissage automatique et les rend peu fiables sur les classes minoritaires.

Ce mini-projet vous propose d'explorer et de comparer différentes stratégies permettant de gérer ce problème, en utilisant le corpus TWIFL — un corpus algérien annoté pour l'opinion et l'émotion — et DziriBERT comme modèle Transformer de référence.

2. Corpus Officiel : TWIFL

TWIFL (An Algerian Corpus and an Annotation Platform for Opinion and Emotion Analysis) est un corpus de tweets algériens annotés manuellement pour l'analyse d'opinion et d'émotions. Il est disponible librement sur HuggingFace sous la référence arbml/Twifil.

Caractéristique	Détail
Lien du téléchargement	https://huggingface.co/datasets/arbml/Twifil
Nombre de tweets	6 000 tweets
Langue	Dialecte algérien (arabe, Arabizi, code-switching arabe-francais)
Source	Twitter (collecte réelle)

- **Tâche unique** : Classification de sentiments (3 classes)
- **Variable cible** : Polarity Class
  Classes : Positive | Negative | Neutral
- **Objectif** : Prédire le sentiment d'un tweet algérien à partir de son texte
- Les autres colonnes (Emotion, User Age, etc.) ne sont pas utilisées dans ce projet.

3. Objectifs Pédagogiques

- ✓ Télécharger et explorer un corpus NLP réel depuis HuggingFace.
- ✓ Analyser et quantifier le déséquilibre de classes dans un dataset de tweets algériens.
- ✓ Appliquer un prétraitement spécifique au dialecte algérien (darija, code-switching, Arabizi).
- ✓ Implémenter et comparer plusieurs stratégies de gestion du déséquilibre de classes.
- ✓ Utiliser DziriBERT pour la classification de sentiments par fine-tuning.
- ✓ Évaluer rigoureusement un modèle avec des métriques adaptées aux données déséquilibrées.
- ✓ Rédiger un rapport scientifique structuré et présenter oralement des résultats.

4. Modele de Base : DziriBERT

Tous les binômes/monômes utilisent DziriBERT comme modèle Transformer de référence. Aucun autre modèle de base n'est autorisé, sauf pour le bonus.

Parametre	Valeur
Identifiant HuggingFace	alger-ia/dziribert
Architecture	BERT (encodeur bidirectionnel)
Tokens d'entrainement	20 millions
Taille du vocabulaire	50 000 tokens
Dimension embeddings [CLS]	768
Langues supportées	Arabe dialectal algerien + Arabizi

5. Protocole Commun

Le protocole suivant est obligatoire. Tout écart devra être justifié par écrit dans le rapport, sous peine de pénalisation.

5.1 Partitionnement des données

Partition	Proportion	Utilisation
Train set	70 %	Entrainement — le rééquilibrage s'applique UNIQUEMENT ici
Validation set	15 %	Choix et ajustement des hyperparametres
Test set	15 %	Evaluation finale — JAMAIS modifie, JAMAIS rééquilibre

Remarque : Le test set est créé une seule fois avec random_state=42 et stratification. Il ne sera jamais rééquilibré, augmenté ou modifié. Toutes les stratégies sont évaluées sur exactement le même test set.

5.2. Hyperparamètres de fine-tuning

Hyperparametre	Valeur imposee
Nombre d'epochs	5
Learning rate	2e-5
Batch size	16
Seed aleatoire	42 (partout dans le code)
Optimiseur	AdamW

5.3. Métriques d'évaluation obligatoires

INTERDIT : L'accuracy ne peut pas être la métrique principale.
Sur des données déséquilibrées, l'accuracy est une métrique trompeuse.
Exemple : si 70 % des tweets sont Positifs, un modèle qui prédit toujours Positif a 70 % d'accuracy mais est complètement inutile. L'accuracy sera mentionnée uniquement pour illustrer ce problème.

Metrique	Description	Pourquoi l'utiliser ?
F1-macro	Moyenne du F1 de chaque classe sans ponderation	Penalise les mauvaises predictions sur les classes minoritaires
F1 par classe	F1 separe pour Positive, Negative, Neutral	Permet de voir quelle classe beneficie de chaque strategie
Precision / Rappel	Par classe separement	Analyse du compromis precision vs rappel
AUC-PR	Aire sous la courbe Precision-Rappel	Plus informative qu'AUC-ROC sur donnees desequilibrees
G-mean	Racine de la moyenne geometrique des rappels	Evalue l'equilibre global entre toutes les classes
Matrice de confusion	Tableau des predictions vs vraies classes	Visualiser les erreurs par classe

6. Etapes Détaillées du Projet

ETAPE 1 — Analyse Exploratoire du Corpus TWIFL

Cette étape est fondamentale. Avant toute modélisation, vous devez comprendre vos données. Elle est notée et conditionnera la qualité de votre analyse finale.

1.1 Analyse de la variable cible

- Calculer le nombre d'exemples par classe (Positive / Negative / Neutral)
- Calculer le ratio de déséquilibre : classe majoritaire / classe minoritaire
- Tracer un bar chart et un pie chart de la distribution

• Conclure : votre corpus est-il fortement, modérément ou faiblement déséquilibré ?

1.2 Analyse linguistique des tweets

• Distribution de la longueur des tweets (en mots et en caractères) — par classe

• Proportion de tweets en arabe / français / Arabizi / langue indéterminée (colonne lang)

• Proportion de tweets contenant des emojis

• Proportion de tweets avec code-switching (mélange arabe + mots français)

• Nuage de mots (WordCloud) séparé par classe de sentiment

1.3 Tableau de statistiques (Obligatoire dans le rapport)

Classe	Nb exemples	% du corpus	Moy. mots	Moy. caracteres
Positive	...	...	...	...
Negative	...	...	...	...
Neutral	...	...	...	...
TOTAL	6 000	100 %	...	...

ETAPE 2 — Prétraitement Spécifique au Dialecte Algérien

Le darija algérien nécessite un prétraitement adapté. N'utilisez pas de pipelines génériques pour l'anglais ou l'arabe standard.

2.1 Nettoyage de base

• Supprimer les URLs (https://t.co/...)

• Supprimer les mentions (@utilisateur)

• Supprimer ou normaliser les emojis — justifier votre choix dans le rapport

• Supprimer les tweets en double

• Supprimer les tweets vides ou sans texte utile

2.2 Normalisation spécifique au darija

• Normaliser les lettres arabes répétées (ex : مزيان → ممزيان)

• Normaliser les chiffres arabes orientaux en chiffres occidentaux si nécessaire

• Ne PAS supprimer les mots français : le code-switching est une caractéristique linguistique du darija

• Ne PAS faire de stemming ni de lemmatisation : DziriBERT tokenise lui-même

2.3 Gestion des cas spéciaux du corpus TWIFL

• Les tweets de langue 'und' (indéterminée) : les conserver et analyser leur contenu

• Les tweets très courts (1-2 mots) : décider si on les conserve ou les filtre (justifier)

• La colonne Polarity Class : vérifier l'absence de valeurs inattendues

2.4 Split train / val / test

Réaliser le split une seule fois avec les pourcentages de répartition en Section 5.1, avant tout rééquilibrage.

Note importante — Flexibilité du prétraitement
Les étapes de prétraitement présentées dans cette section sont des propositions.
Vous êtes libres d'en ajouter, d'en supprimer ou de les modifier selon vos observations sur le corpus TWIFL et selon les besoins de votre approche.

Tout choix de prétraitement doit être justifié dans votre rapport :
— Pourquoi avez-vous ajouté cette étape ?
— Pourquoi avez-vous supprimé ou modifié une étape proposée ?
— Quel impact avez-vous observé sur la qualité des données ?

ETAPE 3 — Modèle Baseline

Le modèle baseline est entraîné sur les données brutes déséquilibrées, sans aucune technique de rééquilibrage. Il constitue la référence que toutes les stratégies doivent améliorer.

1. Charger DziriBERT avec les hyperparamètres imposés (Section 5.2)
2. Fine-tuner sur le train set original (déséquilibré)
3. Évaluer sur le test set et calculer toutes les métriques (Section 5.3)
4. Afficher le rapport de classification complet et la matrice de confusion
5. Sauvegarder ces résultats — ils serviront de référence dans le tableau comparatif final

Point d'attention :

Attendez-vous à ce que la baseline soit biaisée vers la classe majoritaire.

C'est normal et attendu. Votre travail est de montrer ce biais et de le corriger.

Le F1-macro de la baseline est votre score de référence à améliorer.

ETAPE 4 — Stratégie 1 : Modification de la Fonction de Perte

Principe fondamental de cette stratégie

On ne modifie pas les données. Le corpus reste déséquilibré tel quel. On change uniquement la façon dont le modèle apprend à partir de ses erreurs. C'est une approche au **niveau de l'algorithme**, pas au niveau des données.

Par défaut, DziriBERT utilise une CrossEntropy standard qui traite toutes les classes de la même façon. Si 65 % des tweets TWIFL sont Positifs, le modèle apprend à prédire Positif très vite, car cela fait baisser son erreur rapidement. La classe Neutral ou Negative, étant minoritaire, contribue peu à la perte globale et le modèle finit par l'ignorer.La modification de la fonction de perte corrige ce biais en signalant au modèle : **une erreur sur la classe minoritaire coûte plus cher qu'une erreur sur la classe majoritaire.**

4.1 Variante A — Class Weighting

C'est la méthode la plus simple et la plus directe. On calcule un poids inversement proportionnel à la fréquence de chaque classe, puis on l'injecte dans la **CrossEntropy**.

Concrètement, si la classe Neutral représente 10 % du corpus, elle reçoit un poids environ 5 fois plus élevé que la classe Positive qui en représente 50 %. Chaque erreur sur Neutral compte donc 5 fois plus dans le calcul de la perte totale.

Ce que vous devez faire :

- Calculer le poids de chaque classe en utilisant la formule :

poids = total_exemples / (nb_classes × nb_exemples_de_la_classe)

- Injecter ces poids dans la fonction de perte CrossEntropy lors du fine-tuning de DziriBERT
- Garder exactement les mêmes hyperparamètres que la Baseline (lr, epochs, batch size)
- Évaluer sur le test set et comparer les métriques avec la Baseline

4.2 Variante B — Focal Loss

La Focal Loss va plus loin que le Class Weighting. Elle introduit un mécanisme qui réduit dynamiquement la contribution des exemples faciles — ceux que le modèle classifie déjà bien avec une forte confiance — pour concentrer l'apprentissage sur les exemples difficiles.

Elle repose sur un paramètre **gamma (γ)** qui contrôle cet effet :

- Quand $\gamma = 0$: la Focal Loss est identique à la CrossEntropy standard
- Quand $\gamma = 1$: les exemples que le modèle prédit avec 90 % de confiance contribuent peu à la perte
- Quand $\gamma = 2$: l'effet est encore plus prononcé — seuls les exemples vraiment difficiles comptent

Pourquoi c'est pertinent pour le corpus TWIFL ?

Les tweets Positifs, très nombreux, sont souvent faciles à détecter. Le modèle les reconnaît rapidement. La Focal Loss va automatiquement réduire leur poids une fois qu'ils sont bien appris, et forcer le modèle à se concentrer sur les tweets Neutral et Negative plus difficiles à identifier.

Ce que vous devez faire :

- Implémenter la Focal Loss avec le paramètre gamma
- Tester deux valeurs : $\gamma = 1$ et $\gamma = 2$, et comparer les résultats
- Vous pouvez aussi combiner Focal Loss avec Class Weighting pour amplifier l'effet
- Évaluer sur le test set et remplir le tableau comparatif

4.3 Variante C — Combinaison Class Weighting + Focal Loss

Il est possible de combiner les deux approches : appliquer les poids de classe **ET** la Focal Loss en même temps. Les poids corrigent le déséquilibre structurel entre les classes, et la Focal Loss gère la difficulté des exemples individuels au sein de chaque classe. C'est souvent la combinaison qui donne les meilleurs résultats, mais elle nécessite plus de soin dans le calibrage.

4.4 -- Ce que vous devez analyser et inclure dans votre rapport

L'intérêt de cette stratégie est de montrer que **modifier uniquement la fonction de perte** peut suffire à corriger le biais, sans aucune modification des données. Votre analyse doit répondre aux questions suivantes :

- Le **F1** de la classe minoritaire a-t-il augmenté par rapport à la Baseline ? De combien ?
- Y a-t-il un **compromis** ? Le F1 de la classe majoritaire a-t-il baissé quand celui de la minoritaire a augmenté ?
- Quel **gamma** donne les meilleurs résultats sur TWIFL ? Pouvez-vous expliquer pourquoi ?
- **Class Weighting** ou **Focal Loss** : quelle variante est la plus efficace sur ce corpus ?

ETAPE 5 — Stratégie 2 : Rééquilibrage dans l'Espace des Embeddings

Principe fondamental de cette stratégie

Contrairement à la Stratégie 1, on intervient ici sur les **données elles-mêmes**.
L'idée est de générer des exemples synthétiques pour les classes minoritaires afin d'équilibrer la distribution avant l'entraînement.

On ne travaille pas directement sur le texte brut des tweets — le darija est une langue trop complexe pour générer des phrases cohérentes par interpolation.
On travaille dans l'**espace des embeddings de DziriBERT** : chaque tweet est représenté par un vecteur numérique de dimension 768 (le vecteur [CLS]), et c'est dans cet espace que les nouveaux exemples sont générés.

5.1 Le vecteur [CLS] de DziriBERT

Quand DziriBERT traite un tweet, il produit une représentation vectorielle de chaque token. Le token spécial **[CLS]**, placé en début de chaque séquence, capture une **représentation globale du tweet entier**. Ce vecteur de 768 dimensions est ce qu'on appelle **l'embedding du tweet**.

Deux tweets de sentiment similaire auront des vecteurs [CLS] proches dans cet espace. C'est sur cette **propriété géométrique** que reposent toutes les méthodes de rééquilibrage de cette étape : on crée de nouveaux points dans les zones peu peuplées (les classes minoritaires) en interpolant entre des points existants.

Ce que vous devez faire :

- Charger DziriBERT en mode **extraction de features** (sans la tête de classification)
- Pour chaque tweet du **train set**, passer le texte dans DziriBERT et récupérer le vecteur [CLS]
- Stocker ces vecteurs dans une matrice de forme **(N, 768)** où N est le nombre de tweets
- **Ne jamais extraire les embeddings du val set ou du test set** pour le rééquilibrage

5.2 Variante A — SMOTE (Synthetic Minority Over-sampling Technique)

SMOTE est la méthode de référence pour le rééquilibrage. Plutôt que de dupliquer des exemples existants, elle crée de nouveaux exemples synthétiques en interpolant entre des exemples réels de la **classe minoritaire** et leurs voisins les plus proches.

Le processus concrètement : pour chaque exemple de la classe minoritaire, SMOTE identifie ses **k voisins les plus proches** dans l'espace des embeddings DziriBERT. Il choisit aléatoirement un de ces voisins et crée un nouveau point situé quelque part sur le segment qui relie les deux. Le nouveau vecteur est donc une **interpolation linéaire** entre deux exemples réels.

Ce que vous devez faire :

- Appliquer SMOTE sur la matrice d'embeddings du **train set uniquement**
- Tester différents niveaux de rééquilibrage : équilibre total ou partiel
- Entrainer un classifieur (MLP ou SVM) sur les embeddings rééquilibrés
- Évaluer sur le **test set** et comparer avec la Baseline et la Stratégie 1

5.3 Variante B — ADASYN (Adaptive Synthetic Sampling)

ADASYN est une extension de SMOTE qui adapte le nombre d'exemples générés selon la difficulté de chaque exemple. Les exemples de la classe minoritaire qui sont entourés de beaucoup d'exemples de la classe majoritaire — donc difficiles à classifier — recevront plus de nouveaux voisins synthétiques que les exemples déjà bien isolés. En d'autres termes, ADASYN concentre l'effort de génération sur les zones de décision les plus difficiles, là où le modèle a tendance à se tromper.

5.4 -- Ce que vous devez analyser et inclure dans votre rapport

- Travailler dans l'**espace des embeddings** est-il plus efficace que de travailler sur le texte brut ?
- Le **rééquilibrage par embeddings** améliore-t-il le F1-macro plus que la Stratégie 1 ?
- Y a-t-il un **risque de sur-apprentissage** visible sur les courbes de validation ?
- Quelle est l'**influence du niveau de rééquilibrage** choisi (total vs partiel) ?

ETAPE 6 — Stratégie 3 : Back-Translation

Principe fondamental de cette stratégie

La **Back-Translation** est une technique d'augmentation de données qui exploite la **traduction automatique** pour générer des **paraphrases naturelles**.

Le principe : on prend un tweet de la classe minoritaire, on le traduit vers une langue pivot (le français), puis on le retraduit vers l'arabe pour obtenir une **formulation différente du même message**. La phrase résultante exprime le même **sentiment** mais avec d'autres mots et une autre structure. On applique cette technique **uniquement sur la classe minoritaire du train set** pour augmenter son volume sans toucher aux classes bien représentées.

6.1 Les quatre étapes du processus

La **Back-Translation** se déroule en quatre phases successives :

Phase 1 — Sélection :
Identifier tous les tweets appartenant à la **classe minoritaire** dans le train set. C'est uniquement sur cette classe que la Back-Translation sera appliquée.

Phase 2 — Traduction vers le français :
Traduire chaque tweet sélectionné de l'arabe/darija vers le français en utilisant un système de traduction automatique (Helsinki-NLP sur HuggingFace ou une API externe).
Attention : les tweets en Arabizi peuvent poser problème car les systèmes de traduction les reconnaissent mal.

Phase 3 — Retraduction vers l'arabe :
Retraduire les phrases françaises vers l'arabe. On obtient ainsi une **nouvelle formulation** du tweet original en arabe. Cette formulation sera différente de l'original car la traduction n'est pas bijective.

Phase 4 — Filtrage :
Ne pas injecter toutes les paraphrases générées. Calculer la **similarité cosinus** entre chaque paraphrase et le tweet original (en utilisant leurs embeddings DziriBERT).
Conserver uniquement les paraphrases avec une similarité comprise entre **0.5 et 0.85** :

- Trop similaire → pas d'apport nouveau
- Trop différente → le sens a été perdu

6.2 Injection et réevaluation

Après le filtrage, les nouvelles paraphrases sont ajoutées au **train set de la classe minoritaire**. On re-fine-tune ensuite DziriBERT sur ce **train set augmenté** et on évalue sur le **test set figé**.

Il est recommandé de tester différents **taux d'augmentation** et de les comparer :

- Augmentation de 20 % : ajouter 20 % d'exemples supplémentaires par rapport à la taille originale de la classe minoritaire
- Augmentation de 50 % : ajouter la moitié de la taille originale en paraphrases
- Augmentation de 100 % : doubler la taille de la classe minoritaire avec des paraphrases

6.3 -- Ce que vous devez analyser et inclure dans votre rapport

- Quel pourcentage des paraphrases générées passent le filtre de similarité (0.5 - 0.85) ?
- Les paraphrases générées sont-elles en darija ou en arabe standard ? Discutez cette dérive.
- Le taux d'augmentation influence-t-il les performances ? Quel taux est optimal ?

• Avez-vous observé des paraphrases où le sentiment a changé lors de la traduction ? Donnez des exemples.

7. Evaluation Finale et Tableau Comparatif

Tous les modèles sont évalués sur le même test set figé. Le tableau suivant doit être produit dans votre rapport avec vos résultats réels.

Configuration	F1-macro	F1 Positive	F1 Negative	F1 Neutral	AUC-PR	G-mean
Baseline (données brutes)	...	...	...	...	...	...
Class Weighting	...	...	...	...	...	...
Focal Loss (gamma=1)	...	...	...	...	...	...
Focal Loss (gamma=2)	...	...	...	...	...	...
SMOTE + DziriBERT	...	...	...	...	...	...
ADASYN + DziriBERT	...	...	...	...	...	...
Back-Translation	...	...	...	...	...	...

8. Livrables Attendus

Livrable	Format	Echeance
Code Python	Notebooks Jupyter commentes + requirements.txt + README sur GitHub	J-1 avant présentation
Rapport écrit	PDF, 15 à 20 pages (hors annexes)	J-1 avant présentation
Présentation slides	PowerPoint , 15 slides maximum	Jour J (présentation)
Présentation orale	10 min expose + 05 min questions	Jour J (présentation)

Structure du rapport recommandée

1. Introduction et problématique — pourquoi le déséquilibre est un problème (1-2 pages)
2. État de l'art — déséquilibre de classes en NLP, TALN pour le dialecte algérien (2-3 pages)
3. Présentation du corpus TWIFL — EDA, statistiques, nettoyage, caractéristiques linguistiques (2-3 pages)
4. Méthodologie — description claire de chaque stratégie implémentée (3-4 pages)
5. Résultats — tableau comparatif, matrices de confusion, courbes Precision-Rappel (3-4 pages)
6. Discussion — analyse critique des résultats, limites, spécificités du darija (2-3 pages)
7. Conclusion et perspectives (1 page)
8. Références bibliographiques (format IEEE ou APA)
9.

9. Grille de Notation

Critère d'évaluation	Points	Details
Analyse exploratoire (EDA)	1,5 pts	Distribution, visualisations, analyse linguistique, conclusions chiffrées
Preprocessing adapté au darija	1,5 pts	Choix justifies, adaptation au corpus TWIFL, analyse de l'impact sur les données
Baseline correctement implémentée	2 pts	Fine-tuning DziriBERT, résultats cohérents, métriques correctes
Stratégie 1 — Modification de la perte	2 pts	Class Weighting + Focal Loss + combinaison (1 pts chacun)
Stratégie 2 — SMOTE / ADASYN	2 pts	Extraction embeddings + variantes comparées
Stratégie 3 — Back-Translation	1,5 pts	Implémentation, filtrage, réévaluation, analyse des paraphrases
Rapport avec Discussion et analyse critique	2 pts	Réponses aux questions, spécificités du darija, perspectives
Qualité du code	1 pt	Test set intact, métriques imposées, tableau comparatif complet Structure, commentaires, reproductibilité (README + requirements)
Soutenance orale	1,5 pts	Clarté, maitrise du sujet, réponses aux questions
TOTAL	**15 pts**	

Bonus (+2 pts) — Combinaison de stratégies

Les groupes qui implémentent une stratégie hybride originale peuvent obtenir +2 pts.

Exemple : combiner Class Weighting + SMOTE sur les embeddings, ou tester différents ratios d'augmentation (20%, 50%, 100%) et analyser l'impact.

La contribution doit être clairement documentée et analysée dans le rapport.